

Wording for “functionality gaps in constraints”

This paper contains formal wording for the two extensions proposed in [P0766r1: Fixing small-ish functionality gaps in constraints](#) that concern *requires-clauses*.

A Proposed wording for constrained lambdas

Modify the grammar in [expr.prim.lambda, 8.1.5] as follows.

lambda-expression:

```
lambda-introducer compound-statement
lambda-introducer lambda-declaratoropt requires-clauseopt compound-statement
lambda-introducer < template-parameter-list > requires-clauseopt
compound-statement
lambda-introducer < template-parameter-list > requires-clauseopt
lambda-declaratoropt requires-clauseopt compound-statement
```

Modify [expr.prim.lambda, 8.1.5] paragraph 3 as follows.

In the *decl-specifier-seq* of the *lambda-declarator*, each *decl-specifier* shall either be `mutable` or `constexpr`. *[Note: The trailing *requires-clause* is described in Clause [dcl.decl, 11]. – end note]*

Modify [expr.prim.lambda, 8.1.5.1] paragraph 3 as follows.

The closure type for a non-generic *lambda-expression* has a public inline function call operator (16.5.4) whose parameters and return type are described by the *lambda-expression*’s *parameter-declaration-clause* and *trailing-return-type* respectively. For a generic lambda, the closure type has a public inline function call operator member template (17.6.2) whose *template-parameter-list* consists of the specified *template-parameter-list*, if any, to which is appended one invented type *template-parameter* for each occurrence of `auto` in the lambda’s *parameter-declaration-clause*, in order

of appearance. The invented type *template-parameter* is a parameter pack if the corresponding *parameter-declaration* declares a function parameter pack (11.3.5). The return type and function parameters of the function call operator template are derived from the *lambda-expression's trailing-return-type* and *parameter-declaration-clause* by replacing each occurrence of *auto* in the decl-specifiers of the parameter-declaration-clause with the name of the corresponding invented template-parameter. The *requires-clause* of the function call operator template is the *requires-clause* immediately following < *template-parameter-list* >, if any. The trailing *requires-clause* of the function call operator or operator template is the *requires-clause* following the *lambda-declarator*, if any. [Example: [...]

Modify [expr.prim.lambda, 8.1.5.1] paragraph 6 as follows.

The closure type for a non-generic *lambda-expression* with no *lambda-capture* whose constraints (if any) are satisfied has a conversion function to pointer to function with C++ language linkage (10.5) having the same parameter and return types as the closure type's function call operator. The conversion is to "pointer to *noexcept* function" if the function call operator has a non-throwing exception specification. The value returned by this conversion function is the address of a function *F* that, when invoked, has the same effect as [...]

Insert a new paragraph between paragraphs 5 and 6 of [expr.prim.lambda.closure, 8.1.5.1].

[...] - end example]

The function call operator or operator template may be constrained ([temp.constr.decl, 17.4.2]) by a *constrained-parameter* ([temp.param, 17.1]), a *requires-clause* (Clause [temp, 17]), or a trailing *requires-clause* (Clause [dcl.decl, 11]). [Example:

```
template <typename T> concept C1 = /* ... */;
template <std::size_t N> concept C2 = /* ... */;
template <typename A, typename B> concept C3 = /* ... */;

auto f = [<typename T1, C1 T2> requires C2<sizeof(T1) +
sizeof(T2)>
        (T1 a1, T1 b1, T2 a2, auto a3, auto a4) requires
C3<decltype(a4), T2> {
    // T2 is a constrained parameter,
```

```
// T1 and T2 are constrained by a requires-clause, and
// T2 and the type of a4 are constrained by a trailing
requires-clause.
};
```

- end example]

The closure type for a non-generic *lambda-expression* [...]

B Proposed wording for constrained template *template-parameters*

Modify the grammar in [temp.param, 17.1] as follows.

template-parameter:

[...]

type-parameter:

type-parameter-key ..._{opt} *identifier*_{opt}

*type-parameter-key identifier*_{opt} = *type-id*

~~template~~ <~~template-parameter-list~~>*template-head* *type-parameter-key*

..._{opt} *identifier*_{opt}

~~template~~ <~~template-parameter-list~~>*template-head* *type-parameter-key*

*identifier*_{opt} = *id-expression*

type-parameter-key:

[...]

Modify [temp.arg.template, 17.3.3], paragraph 3 as follows.

A *template-argument* matches a template *template-parameter* *P* when *P* is at least as specialized as the *template-argument* *A*. If *P* contains a parameter pack, then *A* also matches *P* if each of *A*'s template parameters matches the corresponding template parameter in the ~~template-parameter-list~~*template-head* of *P*. Two template parameters match if they are of the same kind (type, non-type, template), for non-type *template-parameters*, their types are equivalent (17.6.6.1), and for template *template-parameters*, each of their corresponding *template-parameters* matches, recursively. When *P*'s ~~template-parameter-list~~*template-head* contains a template parameter pack (17.6.3), the template parameter pack will match zero or more template parameters or template parameter packs in the ~~template-~~

~~parameter-list~~template-head of **A** with the same type and form as the template parameter pack in **P** (ignoring whether those template parameters are template parameter packs). [Example: [...]]

Modify [temp.arg.template, 17.3.3], paragraph 4 as follows.

A template *template-parameter* **P** is at least as specialized as a template *template-argument* **A** if, given the following rewrite to two function templates, the function template corresponding to **P** is at least as specialized as the function template corresponding to **A** according to the partial ordering rules for function templates (17.6.6.2). Given an invented class template **X** with the ~~template-parameter-list~~template-head of **A** (including default arguments and requires-clause, if any):

- Each of the two function templates has the same template parameters and requires-clause (if any), respectively, as **P** or **A**.
- Each function template has a single function parameter whose type is a specialization of **X** with template arguments corresponding to the template parameters from the respective function template where, for each template parameter **PP** in the ~~template-parameter-list~~template-head of the function template, a corresponding template argument **AA** is formed. If **PP** declares a parameter pack, then **AA** is the pack expansion **PP . . .** (17.6.3); otherwise, **AA** is the *id-expression* **PP**.

If the rewrite produces an invalid type, then **P** is not at least as specialized as **A**.